

no java before code

Perfect. This file is much smaller than `EmailService`, but the concepts are very important.

Let's start with Part 1 in the same deep-dive style.

◆ GLOBAL EXCEPTION HANDLER CLASS

```
@ControllerAdvice
public class GlobalExceptionHandler {
```

This class is responsible for handling exceptions across the entire application.

Before understanding this class, let's understand the problem it solves.

◆ WHAT IS AN EXCEPTION?

An exception is an error that occurs while the application is running.

Examples:

Flask server is down

↓

Connection error

Wrong URL

↓

404 error

Database unavailable

↓

Connection error

Null object accessed

↓

NullPointerException

All of these are exceptions.

◆ WHAT HAPPENS WITHOUT EXCEPTION HANDLING?

Suppose:

```
emailService.analyze(...)
```

throws an exception.

Example:

```
Connection refused
```

because Flask is not running.

Without exception handling:

Flow

User

↓

Controller

↓

Service

↓

Exception

↓

Application returns ugly error

Example response:

```
{
  "timestamp": "...",
  "status": 500,
  "error": "Internal Server Error"
}
```

This response is difficult for users to understand.

◆ WHAT IS THE GOAL?

Instead of returning:

```
{
  "status": 500,
  "error": "Internal Server Error"
}
```

we want:

```
{
  "result": "SERVER_ERROR",
  "confidence": 0.0
}
```

A clean response.

This class helps us do that.

◆ WHAT IS @ControllerAdvice ?

```
@ControllerAdvice
```

This annotation tells Spring:

This class will handle exceptions for all controllers.

Notice:

ALL controllers.

Not just EmailController.

Think of it as a security guard.

Visual

EmailController

↓

Exception

↓

GlobalExceptionHandler

AnotherController

↓

Exception

↓

GlobalExceptionHandler

Every controller can use it.

◆ WHY IS IT CALLED GLOBAL?

Because it works application-wide.

Suppose tomorrow you create:

UserController

and

AdminController

and

EmailController

If any of them throws an exception:

Spring can send the exception to:

GlobalExceptionHandler

That's why it is called "Global".

◆ WHAT DOES SPRING DO?

When Spring starts:

```
@SpringBootApplication
```

it scans the project.

When it finds:

```
@ControllerAdvice
```

Spring says:

```
This class contains global
exception handling rules.
```

Spring registers it automatically.

You never create an object manually.

Spring manages it.

◆ CLASS STRUCTURE

```
public class GlobalExceptionHandler
```

This is a normal Java class.

The special behavior comes from:

```
@ControllerAdvice
```

Without this annotation:

```
public class GlobalExceptionHandler
```

would just be an ordinary class.

Spring would ignore it.

◆ REAL PROJECT FLOW

Normal Flow

User

↓

Controller

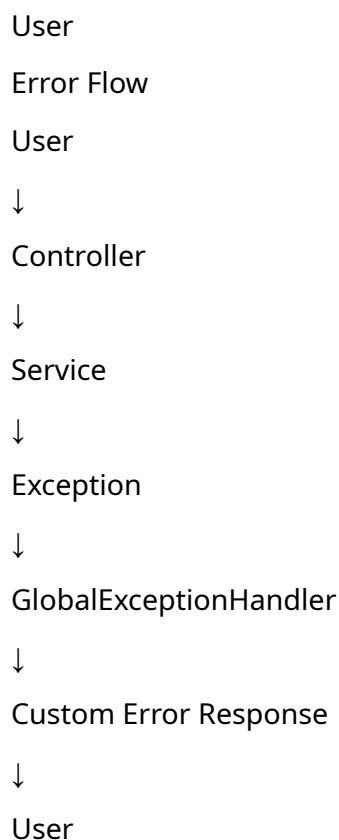
↓

Service

↓

Response

↓



◆ WHY IS THIS USEFUL?

Imagine Flask is down.

Instead of:

```
Connection refused
```

causing an ugly crash response,
we can return something meaningful like:

```
SERVER_ERROR
```

This creates a better API experience.

◆ SUMMARY

```
@ControllerAdvice
```

marks this class as a global exception handler.

An exception is an error that occurs while the application is running.

Without exception handling, users may receive ugly default error messages.

With:

```
GlobalExceptionHandler
```

Spring can intercept exceptions and return clean, consistent responses.

The next section should continue with:

```
@ExceptionHandler(Exception.class)
public ResponseEntity<EmailResponseDTO>
handleException(Exception e)
```

because this is where Spring learns which exceptions to catch and what method should handle them.